v2 · Enhanced

THENOTHING V7.1 · CLAUDE CODE MODE

HYDRA Offensive Knowledge Operating System

Project Documentation — Phase O: Temporal Knowledge Intelligence

15 phases (A–O) • **153** effective capabilities • **439** adapters • **7** agents • **108** MCP tools

Install & Quick Start | Usage Examples | Wiki Schema | Promotion Workflow | Offensive Memory | Contributing | Troubleshooting | 499 tests green

Comprehensive architecture + practical guide | Generated 2026-06-06
Single source of truth: docs/HYDRA_SYSTEM_CONTEXT.md (architecture memory)

Table of Contents

1 Project Overview

HYDRA is an **Offensive Knowledge Operating System** — a layered, pure-Python platform that an LLM operator drives through the **Model Context Protocol (MCP)** to perform high-quality bug-bounty and offensive-security **research within explicit authorization**. It is the engine behind **THENOTHING v7.1 — Claude Code Mode**.

The platform is built bottom-up across **15 locked phases (A → O)**. Each phase adds a *derived, advisory* intelligence layer on top of a single **canonical knowledge store** (the wiki). Nothing below the wiki ever writes back to it except through explicit, propose-only paths. Every derived layer is deterministic, offline-first, and rebuild-identical.

Property	Value
Current Phase	**O — Temporal Knowledge Intelligence**
Core capabilities	87
Effective capabilities	153 (core + 6 plugin packs)
Adapters	175 core / 439 effective
Agents	7
Plugins	6 reference packs
MCP tools	108
Tests	499 passing (6 integration deselected)
License	MIT

> **Design philosophy:** reason before executing, simulate before interacting, correlate evidence across domains, learn from every outcome — while never silently mutating the canonical wiki, the promotion rules, or the confidence engine.

2 Installation & Setup

System requirements

Requirement	Detail
Python	≥ 3.10 (3.13 recommended)
OS	Linux / Kali (macOS & WSL supported); offline-first core needs no external services
Disk	Modest — derived stores under `data/` are small SQLite files
Recon tooling (optional)	subfinder, httpx, nuclei, sqlmap, ffuf, katana, gau, dnsx... (only for *live* recon)
MCP client	Claude Code, Cursor, or Cline (any MCP-compatible client)

Clone & install

BASH

```
git clone <your-fork-url> hydra
cd hydra

pip install -r requirements.txt        # runtime
pip install -r requirements-dev.txt    # tests, linters

python -m pytest -q                    # 499 passing (6 integration deselected)
```

Run the MCP server

All 108 tools are exposed through the **hydra-security** MCP server.

BASH

```
# stdio (local clients such as Claude Code — auto-loaded via .mcp.json)
python mcp_server.py

# remote / SSE transport (HTTP)
python mcp_server.py --transport sse --port 8900

# verify which optional recon tools are installed
#   (MCP tool) check_tools
```

Claude Code auto-loads the server from `.mcp.json`:
`{ "mcpServers": { "hydra-security": { "command": "python", "args": ["mcp_server.py"] } } }`

Environment variables

Variable	Purpose	Default
HYDRA_WIKI_DIR	Canonical wiki root	`./wiki`

Variable	Purpose	Default
HYDRA_SOURCE_KEYS	Comma-separated API keys enabling online recon sources	(unset = offline)
HYDRA_TOR	Wrap outbound tooling through Tor (1/0)	`0`
HYDRA_PLUGIN_DIR	Directory of declarative plugin packs	`hydra/plugins/packs`
HYDRA_ENFORCE_SCOPE	Hard scope enforcement for live actions	on
HYDRA_*_DB	Override any derived store path (SOURCE_LEARNING, VERIFICATION, TOOL_HEALTH, PLUGIN_HEALTH, DECISION, GOVERNANCE, WORKFLOWS, FEDERATION, TEMPORAL)	`data/*.db`

Load the wiki & initialize databases

The canonical wiki ships under `wiki/`. Derived databases are created automatically on first use (WAL mode) — there is no migration step. To (re)build the derived graph index from the canonical wiki, or to seed knowledge:

MCP TOOLS

```
# rebuild the derived graph index from the canonical wiki  (MCP tool)
kb_rebuild_index

# health-check the wiki (orphans, dangling links, type breakdown)
kb_lint

# fuse recon sources into Asset Intelligence (writes the wiki, offline-first)
recon_fuse domain=example.com

# distill a disclosed report into report+intel pages
ingest_report path=output/example/report.md target=example
```

> Every `data/*.db` store is **derived, disposable, and gitignored**. Delete any of them and it rebuilds identically from its event log — the canonical wiki is the only source of truth.

3 Getting Started (Quick Start)

Once the MCP server is registered, you interact with HYDRA by calling tools from your MCP client. A typical first session follows the cognitive loop: **recall → plan → observe → simulate → act → learn**.

First five commands

MCP SESSION

```
# 1) RECALL — what do we already know? (offensive memory, search-first)
kb_recall query="idor account takeover"

# 2) PLAN — a learning-driven recon plan for a target
recon_plan target=example.com

# 3) OBSERVE — fuse passive recon sources into Asset Intelligence
recon_fuse domain=example.com

# 4) ROUTE — which agent/capability/tool handles this surface?
agent_plan target=example.com type=web_app

# 5) LEARN — how is our knowledge trending over time?
temporal_summary
```

How interaction works (MCP)

```
LLM operator (Claude Code / Cursor / Cline)
      |  calls a tool by name + args
      v
hydra-security MCP server (mcp_server.py)  -- 108 tools
      |  reads/writes ONLY: canonical wiki (propose-only) + derived data/*.db
      v
returns structured JSON  -->  the operator reasons on it and calls the next tool
```

Tools are **read-only by default**. Only a small, explicit set of writers ever touch the canonical wiki (`recon_fuse` , `ingest_report` , `confirm_candidate` , `kb_promote`). Everything else advises.

4 Usage Examples

Illustrative calls and (abridged) JSON responses for the most important tools.

1) Recon Knowledge Fusion — recon_fuse

Collects from policy-allowed sources, dedups, scores confidence by the Two-Signal rule, and writes canonical Asset Intelligence to the wiki.

REQUEST -> RESPONSE

```
recon_fuse domain=example.com capability=discover_subdomains online=false

{
  "domain": "example.com",
  "assets_written": 42,
  "two_signal_high_confidence": 11,
  "sources_used": ["crtsh", "wayback", "dnsx"],
  "wiki_pages": ["assets/api-example-com", "assets/dev-example-com"]
}
```

2) Offensive Memory — kb_recall

REQUEST -> RESPONSE

```
kb_recall query="graphql introspection auth bypass" limit=3

{ "hits": [
  {"slug": "techniques/progressive-auth-probing", "type": "technique", "score": 0.88},
  {"slug": "intel/viator-expapikey-disclosure", "type": "intel", "score": 0.71},
  {"slug": "patterns/graphql-sigv4-leak", "type": "pattern", "score": 0.64} ] }
```

3) Agents — agent_plan & agent_route

REQUEST -> RESPONSE

```
agent_plan target=example.com type=web_app
  -> priority-ordered workflow: recon_agent -> attack_surface_agent ->
     verification_agent -> correlation_agent -> reporting_agent

agent_route target=api.example.com type=api
{ "agent": "attack_surface_agent",
  "capability": "api_endpoint_discovery",
  "tool": "katana",            // learning-selected
  "reasoning": "api surface; katana highest tool-health for this capability" }
```

4) Temporal Intelligence (Phase O) — temporal_forecast & temporal_decay

REQUEST -> RESPONSE

```
temporal_trends domain=capability
  -> [{"entity":"subdomain_discovery","direction":"rising","slope":0.42}, ...]
```

```
temporal_forecast domain=verification horizon=3
{ "verification_coverage": {"slope": 0.18,
    "projection": [4.0, 4.2, 4.4], "projected_next": 4.0},
  "method": "moving_average + linear_slope (bounded, deterministic)" }

temporal_decay
{ "decay_findings": [
   {"entity":"wordpress_xmlrpc","type":"capability","severity":"high",
    "rationale":"no activity for 24 buckets",
    "suggested_action":"review whether still relevant or re-exercise"} ] }
```

5) Report Intelligence — ingest_report

REQUEST -> RESPONSE

```
ingest_report path=output/vk/idor-writeup.md target=vk

{ "success": true,
  "report_page": "reports/vk-idor-2026",
  "intel_page": "intel/vk-idor-rootcause",
  "learning_score": 8,
  "vuln_class": "idor",
  "unresolved_references": ["techniques/object-ref-enumeration"] }
```

6) Pattern Discovery — discover_patterns & confirm_candidate

REQUEST -> RESPONSE

```
discover_patterns                # dry-run; writes nothing
  -> candidate: "cors-misconfig-on-subsidiary" (2 independent weighted evidences)

confirm_candidate id=cors-misconfig-on-subsidiary   # the ONLY write path
  -> wiki page patterns/cors-misconfig-on-subsidiary (status: candidate, provenance)
```

7) Decision Simulation — simulate_workflow

REQUEST -> RESPONSE

```
simulate_workflow target=example.com type=web_app
{ "predicted_findings": 3.4, "verification_success": 0.62,
  "completion_probability": 0.81, "new_pattern_probability": 0.27 }
```

8) Governance Health — governance_summary

REQUEST -> RESPONSE

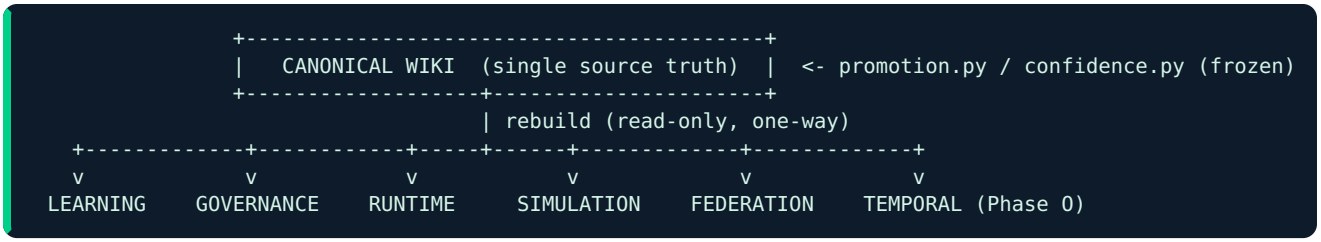
```
governance_summary
{ "knowledge_health_score": 78,
  "drift": {"count": 4, "by_severity": {"low": 3, "medium": 1}},
  "decision_intelligence": { ... },
  "temporal_intelligence": {"temporal_health_score": 71, "status": "healthy"} }
```

> Outputs above are **illustrative** shapes. Exact fields are defined by each tool; all are deterministic given the same inputs.

5 Architecture Highlights

HYDRA's nine-phase reasoning loop sits on a stack of derived, deterministic subsystems:

> **Reasoning loop:** Observe -> Understand -> Reason -> Simulate -> Plan -> Execute -> Validate -> Learn -> Replan



- **Capability-first.** Everything is a *capability* (e.g. `subdomain_discovery`), realized by interchangeable *tools* via *adapters*.
- **Derived & rebuildable.** Every learning/intelligence store is a pure function of an append-only event log; delete it and it rebuilds identically.
- **Advisory-only.** Simulation, governance, federation and temporal layers *recommend*; they never execute, confirm, promote, or exploit.
- **Offline-first & deterministic.** Given a fixed injected clock, outputs are byte-identical (rebuild-identical).

6 Architecture Lineage (Phases A -> O)

Build order is **locked A -> O**. Every phase is derived/advisory unless noted; none mutate promotion.py, confidence.py, or canonical wiki behavior.

Phase	Theme	Purpose	MCP Δ
A	Offensive Knowledge OS Foundations	Capability-first recon + a machine-operable wiki as the single canonical store.	+10
B	Report Intelligence	Distill disclosed reports/writeups into reusable, scored attacker knowledge.	+3
C	Pattern & Chain Discovery (propose-only)	Cross-document synthesis into pattern/chain candidates.	+3
C.5	Scalability Hardening	Remove O(F^2) chain discovery; rebuild amplification fixes.	0
D / D.1	Source Performance Learning & Opportunity Ranking	Event-sourced learning to prioritize without touching canonical state.	+4
E	Adaptive Recon & Autonomous Source Selection	Use Phase-D learning to advise recon planning.	+2
F	Verification Learning & Validation Intelligence	Learn how findings get validated; advisory verification playbooks.	+4
G	Capability Expansion & Tool Orchestration	Capability-centric catalog v2 (87 caps / 9 categories) + learning tool selector.	+4
H	Multi-Agent Orchestration	Declarative agents over the capability layer; deterministic routing.	+4
I	Execution Runtime & Workflow Engine	Deterministic workflow STATE coordination (no execution). Adds mobile_agent -> 87/87 owned.	+4
J	Knowledge Governance, Drift & QA	Derived health/freshness/consistency evaluation.	+6
K	Adapter Framework & Sandboxed Tool Integrations	Capability x tool adapter definitions (175 core) + sandboxed runtime + tool-health learning.	+6
L	Autonomous Knowledge Simulation & Decision Intelligence	Predict workflow/plan/strategy outcomes from historical learning before execution.	+8
M	Capability Marketplace & Plugin Ecosystem	Declarative plugins extend capabilities/adapters/ agents. core(87)+plugins(66)=153 effective, 439 adapters.	+12
N	Federated Knowledge Exchange & Intelligence Mesh	Exchange anonymized, aggregated intelligence digests between instances - metadata only.	+10

Phase	Theme	Purpose	MCP Δ
O	Temporal Knowledge Intelligence	Understand how knowledge evolves over time: trends, momentum, decay, forecasts, anomalies.	+6

Per-phase detail

Phase A — Offensive Knowledge OS Foundations

MCP +10

Purpose. Capability-first recon + a machine-operable wiki as the single canonical store.

Components. CapabilityRegistry, WikiStore, recon-fusion (Two-Signal confidence), graph index, promotion.py, confidence.py, safety/test harness.

Invariants preserved. Wiki canonical; promotion/confidence introduced and frozen thereafter.

Phase B — Report Intelligence

MCP +3

Purpose. Distill disclosed reports/writeups into reusable, scored attacker knowledge.

Components. ReportIntelligencePipeline; report+intel pages only; deterministic 1-10 learning score.

Invariants preserved. No findings/patterns/chains created; LLM-free deterministic scoring.

Phase C — Pattern & Chain Discovery (propose-only)

MCP +3

Purpose. Cross-document synthesis into pattern/chain candidates.

Components. PatternDiscovery, ChainDiscovery, evidence weighting, confirm_candidate (only write path).

Invariants preserved. Discovery is dry-run; canonical pages only via explicit confirm.

Phase C.5 — Scalability Hardening

MCP 0

Purpose. Remove O(F^2) chain discovery; rebuild amplification fixes.

Components. Bounded chain discovery; canonical signatures from structured fields only.

Invariants preserved. Determinism preserved; complexity reduced to O(E).

Phase D / D.1 — Source Performance Learning & Opportunity Ranking

MCP +4

Purpose. Event-sourced learning to prioritize without touching canonical state.

Components. SourceLearningStore, OpportunityScorer; D.1 robustness (no dead-zones).

Invariants preserved. Learning derived/rebuildable; never touches promotion/confidence.

Phase E — Adaptive Recon & Autonomous Source Selection

MCP +2

Purpose. Use Phase-D learning to advise recon planning.

Components. AdaptiveSourceSelector, ReconPlanner.

Invariants preserved. Advisory; recommends, never executes/confirms/writes.

Phase F — Verification Learning & Validation Intelligence

MCP +4

Purpose. Learn how findings get validated; advisory verification playbooks.

Components. VerificationLearningStore, ValidationIntelligence, PlaybookGenerator, ToolCapabilityRegistry.

Invariants preserved. Never auto-confirms/auto-exploits; WAL, idempotent.

Phase G — Capability Expansion & Tool Orchestration

MCP +4

Purpose. Capability-centric catalog v2 (87 caps / 9 categories) + learning tool selector.

Components. CapabilityCatalog, CapabilityCoverage, ToolSelector.

Invariants preserved. Capability modeling only; no integrations; read-only.

Phase H — Multi-Agent Orchestration

MCP +4

Purpose. Declarative agents over the capability layer; deterministic routing.

Components. AgentRegistry (6 agents), AgentPlanner; Target->Agent->Capability->Tool.

Invariants preserved. Agents never execute/confirm/write; advisory.

Phase I — Execution Runtime & Workflow Engine

MCP +4

Purpose. Deterministic workflow STATE coordination (no execution). Adds mobile_agent -> 87/87 owned.

Components. RuntimeEngine, WorkflowStore (workflows.db).

Invariants preserved. Executes no tools; materializes nothing canonical.

Phase J — Knowledge Governance, Drift & QA

MCP +6

Purpose. Derived health/freshness/consistency evaluation.

Components. DriftDetector, KnowledgeQualityAnalyzer, GovernanceIntelligence (knowledge_governance.db).

Invariants preserved. Read-only; writes nothing canonical.

Phase K — Adapter Framework & Sandboxed Tool Integrations

MCP +6

Purpose. Capability x tool adapter definitions (175 core) + sandboxed runtime + tool-health learning.

Components. AdapterRegistry, ToolHealthStore, AdapterIntelligence, CapabilityExerciseAnalyzer.

Invariants preserved. No offensive execution; only SAFE profiles; unsupported profiles rejected at load.

Phase L — Autonomous Knowledge Simulation & Decision Intelligence

MCP +8

Purpose. Predict workflow/plan/strategy outcomes from historical learning before execution.

Components. SimulationContext (load-once O(E)), WorkflowSimulator, OutcomePredictor, PredictionAnalytics.

Invariants preserved. Advisory; governance gains decision_intelligence block; no execution/promotion.

Phase M — Capability Marketplace & Plugin Ecosystem

MCP +12

Purpose. Declarative plugins extend capabilities/adapters/agents. core(87)+plugins(66)=153 effective, 439 adapters.

Components. PluginRegistry, EffectiveCapabilityCatalog, CapabilityDependencyGraph, AgentOwnershipResolver.

Invariants preserved. Globally-unique capability ids; no plugin execution; promotion/confidence untouched.

Phase N — Federated Knowledge Exchange & Intelligence Mesh

MCP +10

Purpose. Exchange anonymized, aggregated intelligence digests between instances - metadata only.

Components. federation/: safety guard, KnowledgeExchangeStore, FederationRegistry, KnowledgeDigestGenerator, IntelligenceMesh, ConsensusEngine, FederationMarketplace.

Invariants preserved. Metadata-only (assert_safe on export+import); advisory; promotion/confidence untouched.

Phase O — Temporal Knowledge Intelligence

MCP +6

Purpose. Understand how knowledge evolves over time: trends, momentum, decay, forecasts, anomalies.

Components. temporal_intel/: TemporalStore, TemporalContext (load-once, memoized), TrendAnalyzer, MomentumAnalyzer, TemporalForecastEngine, DecayAnalyzer, TemporalAnomalyDetector, TemporalAdvisor, TemporalIntelligence.

Invariants preserved. Derived/advisory/deterministic; no wiki mutation; promotion/confidence untouched.

7 The Seven Agents

Introduced in **Phase H** (declarative agents) and made executable-as-*state* in **Phase I** (runtime), the agents form a deterministic routing layer. A target is routed to the right **agent**, which owns a set of capability **categories**, each realized by a learning-selected **tool/adapter**. Agents **never execute tools, confirm findings, or write the wiki** — they plan and route; the runtime tracks state only.

Agent	Priority	Owns categories	Responsibility
recon_agent	10	reconnaissance, infrastructure	Discover and map the external attack surface - subdomains, DNS, ASN/CIDR, ports, services, tech, TLS.
attack_surface_agent	8	web, api	Enumerate the web/API surface and probe for vulnerability candidates (advisory).
cloud_agent	7	cloud, secrets, source_code	Discover cloud assets, leaked secrets, and source-code / dependency exposures.
verification_agent	6	verification	Validate suspected findings using verification playbooks - never auto-confirms.
mobile_agent	6	mobile	Static-analyze mobile apps (APKs): secret/ endpoint extraction, deeplink and cert-pinning checks. Added in Phase I - closes capability ownership to 87/87.
correlation_agent	5	(cross-cutting)	Correlate findings and report-intel into patterns and chains - propose-only; promotion rules unchanged.
reporting_agent	3	(cross-cutting)	Synthesize confirmed knowledge into structured bug-bounty reports.

Expected outputs per agent

Agent	Expected output asset types
recon_agent	subdomain, dns_record, ip, asn, open_port, service, technology, host
attack_surface_agent	endpoint, parameter, url, xss, sqli, ssrf, graphql, vulnerability
cloud_agent	cloud_bucket, cloud_misconfig, secret, credential, repository, dependency_vuln
verification_agent	idor, ssrf, auth_bypass, open_redirect, csrf, xss, sqli, takeover
mobile_agent	mobile_finding, secret, credential, endpoint, internal_host, deeplink, cert_pinning
correlation_agent	pattern, chain
reporting_agent	report

How agents interact with the stack

```
Target --> agent_route --> Agent --> Capability (category-owned) --> Tool (learning-selected)
                                |                                          |
                                v                                          v
                         Adapter (SAFE profile)                  tool_health learning
                                |
                                v
                       Runtime Engine (workflow STATE only - no execution)
```

Routing is deterministic: `agent_route` maps Target→Agent→Capability→Tool. Quality is observable via `agent_coverage` (orphans / overlaps / bottlenecks) and `agent_effectiveness` (Phase-L multi-agent simulation). The effective catalog is **153/153 owned, with zero gaps or conflicts**.

8 MCP Tools (108)

All tooling is exposed to the LLM operator through the **hydra-security** MCP server (`python mcp_server.py`, stdio or SSE). A committed contract baseline and a CLAUDE.md doc-sync test guard against registry drift.

Layer (phase)	Representative tools
Recon & Surface	`subfinder_scan, httpx_probe, katana_crawl, gau_urls, dnsx_resolve, amass_enum, nmap_scan, full_recon, check_tools`
Vulnerability & Fuzzing	`nuclei_scan, nuclei_scan_list, sqlmap_scan, dalfox_scan, gxss_check, ffuf_fuzz, dirsearch_scan`
Fingerprinting	`whatweb_detect, wafw00f_detect`
Knowledge OS (A-C)	`recon_fuse, kb_recall, kb_lint, kb_promote, kb_rebuild_index, asset_lookup, graph_neighbors, graph_path, ingest_report, report_lookup, list_reports, discover_patterns, discover_chains, confirm_candidate`
Learning & Ranking (D-F)	`record_outcome, source_scores, rank_opportunities, prioritization_report, select_sources, recon_plan, record_verification, verification_stats, verification_playbook, tool_capabilities`
Capabilities & Agents (G-H)	`capability_catalog, capability_coverage, rank_tools, select_tool, agent_catalog, agent_plan, agent_route, agent_coverage`
Runtime & Governance (I-J)	`workflow_create, workflow_status, workflow_history, runtime_summary, governance_summary, drift_report, knowledge_health, stale_entities, duplicate_patterns, contradiction_report`
Adapters & Simulation (K-L)	`adapter_catalog, adapter_coverage, adapter_health, adapter_summary, adapter_select, runtime_analytics, simulate_workflow, simulate_strategy, predict_outcome, capability_impact, prediction_accuracy, agent_effectiveness, workflow_optimization, decision_health`
Marketplace (M)	`plugin_catalog, plugin_summary, plugin_health, plugin_dependencies, plugin_capabilities, plugin_coverage, capability_graph, dependency_paths, critical_capabilities, agent_ownership, ownership_conflicts, ecosystem_summary`
Federation (N)	`federation_peers, federation_summary, export_digest, import_digest, capability_trends, verification_trends, source_trends, federation_consensus, ecosystem_opportunities, federation_health`
Temporal (O)	`temporal_summary, temporal_trends, temporal_forecast, temporal_decay, temporal_anomalies, temporal_health`

Beyond the Knowledge-OS phase tools (A-O), the registry also includes the legacy operational palette (recon/scan/fuzz) and base tools (save_finding, get_findings, generate_report). Live total: **108**.

9 Capabilities, Adapters & Plugins

Capabilities

A **capability** is an abstract objective (e.g. `subdomain_discovery`, `port_scanning`, `xss_probing`) independent of any specific tool. The core catalog holds **87** capabilities across 9 categories; six declarative plugin packs raise the **effective** catalog to **153**, with globally-unique ids (duplicates rejected).

Adapters

An **adapter** binds one capability to one tool (`capability::tool`) with a sandboxed, deterministic definition: execution profile, timeouts, I/O schemas. Only **SAFE profiles** (offline / passive / validation / simulation) load; exploitation / persistence / destructive / weaponized profiles are rejected. The effective adapter count is **439** (175 core).

Plugins (6 reference packs)

Pack	Adds
cloud	cloud asset / misconfig capabilities
mobile	mobile (APK) analysis capabilities
container	container / K8s exposure capabilities
iot	IoT surface capabilities
supply_chain	dependency / supply-chain capabilities
osint	OSINT enrichment capabilities

> **core_capabilities + plugin_capabilities = effective_capability_catalog.** A capability dependency graph (requires / enhances / related_to) is acyclic-validated; agent ownership is automatic and complete (153/153).

10 Wiki Structure & Schema

The wiki is the **single canonical knowledge store** — the *synthesis layer* between raw engagement data and the operator. It follows the **LLM-Wiki pattern** and is owned and maintained by the LLM, never a dumping ground for scan output (that stays in `output/`).

The three layers

Layer	Where	Mutability
Raw sources	`output/<program>/`, scope.txt, APK extractions, disclosed reports	Immutable — read, never edit
The wiki	`wiki/`	Created & maintained by the LLM
The schema	`wiki/SCHEMA.md`	Co-evolved by LLM + operator

Folder structure

```
wiki/
|-- SCHEMA.md        # the config layer - read FIRST every session (conventions + reader contract)
|-- index.md         # content catalog / hub - read SECOND to locate pages
|-- log.md           # append-only change log
|-- _templates/      # page templates: target, asset, technique, intel, hypothesis,
|                    #   observation, finding, pattern, chain, report
|-- targets/         # programs/assets under test (hub pages)
|-- assets/          # discovered assets (subdomains, hosts, endpoints) + Asset Intelligence
|-- techniques/      # reusable attacker methodology (human-readable knowledge)
|-- intel/           # distilled, cross-linked report intelligence (root causes, lessons)
|-- hypotheses/      # exploit theories awaiting validation
|-- findings/        # VALIDATED findings (two independent signals)
|-- patterns/        # recurring lessons synthesized across findings
|-- chains/          # composable multi-step attack paths
'-- reports/         # ingested disclosed reports + submitted report links
```

index.md & SCHEMA.md

`SCHEMA.md` is the **configuration layer**: it defines conventions and a non-negotiable **reader contract** — any agent must *store, understand, keep in sync, and verify-before-relying-on* the wiki at the start of every session. `index.md` is the **content catalog** (hub) listing targets, techniques, patterns and chains so a session can locate pages before drilling in.

Page types (NodeTypes) and frontmatter

NodeType	Meaning
target	A program/asset under test; parent hub for discovered assets.
asset	A discovered subdomain/host/endpoint with Two-Signal confidence + sources.
technique	Reusable methodology (the human-readable counterpart of a skill).

NodeType	Meaning
intel	Distilled, reusable knowledge extracted from a disclosed report.
observation	A raw signal compiled from output/ (lowest stage).
hypothesis	An exploit theory with a reasoning trace, awaiting validation.
finding	A validated vulnerability (requires two independent signals).
pattern	A recurring lesson synthesized across two or more independent findings.
chain	A composable multi-step attack path (e.g. SSRF->Admin->RCE).
report	An ingested disclosed report or a submitted report reference.

Every page is Markdown with YAML frontmatter (`type`, `stage`, `confidence`, `tags`, timestamps) and `[[wikilinks]]` that form the knowledge graph:

WIKI/FINDINGS/VK-IDOR-ACCOUNT-SETTINGS.MD

```
---
type: finding
stage: finding
confidence: high
tags: [idor, vk, validated]
created: '2026-06-05'
updated: '2026-06-05'
---

# vk-idor-account-settings

> Validated IDOR on the account-settings endpoint. Two signals: replayed request +
> cross-account read confirmed.

## Evidence
- [[assets/api-vk-com]]
- [[techniques/object-ref-enumeration]]
```

11 Knowledge Promotion Workflow

Knowledge is never born as a 'finding'. It **earns** its stage by climbing a strict ladder, one step at a time, forward only. Validation is mandatory: the engine (`promotion.py`, frozen since Phase A) rejects any shortcut.

The promotion ladder

```
OBSERVATION  ->  INTEL  ->  HYPOTHESIS  ->  FINDING  ->  PATTERN  ->  CHAIN
(raw signal)   (distilled (exploit       (VALIDATED:   (recurring   (multi-step
                lesson)    theory)        2 signals)    lesson)      attack path)

^------------------ promotion is ONE STEP AT A TIME, forward only ------------------^
Two independent signals REQUIRED to enter: FINDING, PATTERN, CHAIN
```

Forbidden transitions (rejected even WITH evidence)

Transition	Why it is blocked
`observation -> finding`	Must pass through intel/hypothesis + validation first
`observation -> pattern / chain`	Cannot skip validation
`intel -> finding`	Intel informs, but a hypothesis must be validated into a finding
`intel -> pattern / chain`	Patterns/chains are built from validated findings only
`hypothesis -> pattern / chain`	A hypothesis must become a finding before informing either

Concrete before / after

A theory about an IDOR starts as a **hypothesis**. Only after two independent signals confirm it does `apply_promotion` move it to **finding** and elevate confidence to **high**:

STEP 1 - HYPOTHESIS

```
# BEFORE - a hypothesis page (unvalidated theory)
---
type: hypothesis
stage: hypothesis
confidence: low
---
# vk-idor-account-settings
> THEORY: account-settings may expose other users' objects via numeric id.
> Falsified if cross-account read returns 403.
```

STEP 2 - VALIDATED FINDING

```
# AFTER - promoted to a finding (two independent signals)
#   apply_promotion(page, to=FINDING, sources=[replay, cross_account_read], evidence_count=2)
---
type: finding
stage: finding
confidence: high          # elevated only with 2 independent signals (Two-Signal rule)
---
```

```
# vk-idor-account-settings
> VALIDATED: id=1337 returned victim's settings; replay + cross-account read confirm.
```

Once several independent findings share a root cause, the `correlation_agent` proposes a **pattern** (via `discover_patterns`), and composable findings become a **chain** — but only through the explicit `confirm_candidate` write path. **promotion.py and confidence.py are never modified.**

12 Offensive Memory System

HYDRA's **Offensive Memory** is the discipline of *recalling before planning*. Before any operation, the operator searches the canonical wiki + derived graph index for prior knowledge — so the system compounds instead of repeating work or re-making known mistakes.

Recall-first workflow

OFFENSIVE MEMORY - MCP

```
# 1) search prior knowledge (techniques, intel, patterns, findings, lessons)
kb_recall query="cors subsidiary takeover"

# 2) inspect a specific asset's accumulated intelligence
asset_lookup slug=api-example-com

# 3) walk the knowledge graph from a page
graph_neighbors slug=techniques/progressive-auth-probing

# 4) find the shortest attack path between two nodes
graph_path from=assets/api-example-com to=chains/ssrf-to-admin
```

Why it matters (the reader contract)

The wiki's `SCHEMA.md` binds every session to a non-negotiable contract: **store** the relevant knowledge into context, **understand** it (why a technique works, which lessons forbid certain framings), **keep it in sync** (write back durable learnings), and **verify before relying** (re-confirm a host/key/version still holds). Knowledge that is read but not internalized — or learned but not written down — is considered lost.

> Offensive memory also spans the **operator memory** (`.claude/.../memory/` , cross-session preferences & rejections) and **skills** (executable methodology). The wiki *embodies* those lessons as reusable technique/pattern pages and links back conceptually.

13 Stores & Databases

Every store under `data/` is **derived, disposable, gitignored**, and rebuildable from its append-only event log (WAL mode, idempotent writes). None is a second canonical source.

Layer	Database	Purpose
Learning	`source_learning.db`	Source performance learning (trust / effectiveness / novelty)
Learning	`source_metrics.db`	Per-source run metrics
Learning	`verification_learning.db`	Verification method / evidence success learning
Learning	`tool_health.db`	Adapter tool-health (reliability / runtime / outcomes)
Learning	`plugin_health.db`	Plugin/capability usage learning
Intelligence	`decision_learning.db`	Simulation / forecasting / strategy (Phase L)
Intelligence	`temporal.db`	Temporal evolution: trends / decay / forecasts (Phase O)
Runtime	`workflows.db`	Deterministic workflow state (Phase I)
Governance	`knowledge_governance.db`	Health / drift snapshots (Phase J)
Federation	`federation.db`	Append-only metadata-only exchange ledger (Phase N)
Canonical index	`knowledge_index.db`	Derived graph index (rebuildable from the wiki)

14 Architecture Invariants

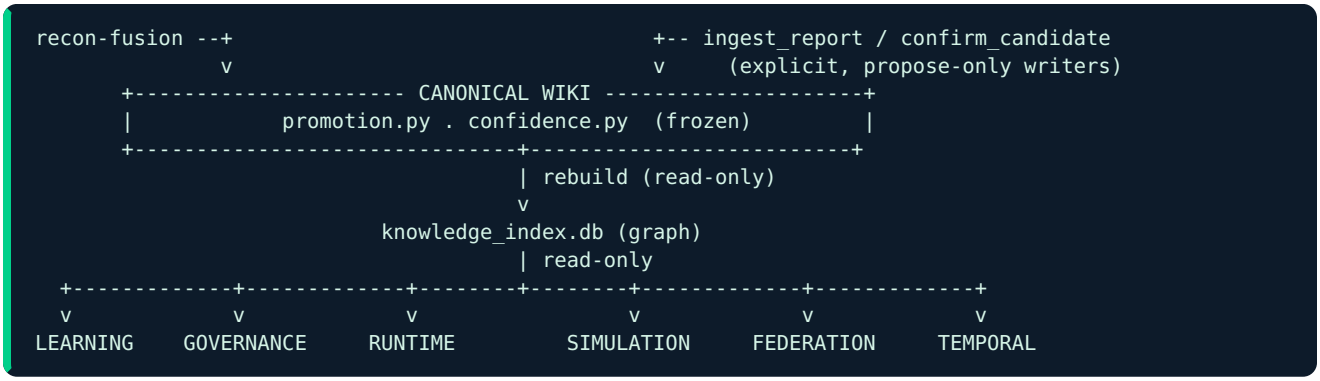
These invariants are enforced in code and **continuously verified by the test suite** (e.g. promotion/confidence immutability checks, no-wiki-mutation tests, metadata-only federation guards, rebuild-identical determinism tests).

Area	Invariant
Canonical knowledge	Wiki is the single canonical source; no second canonical source; no dual-write.
Protected core	promotion.py and confidence.py are immutable (last touched in Phase A).
Discovery	Propose-only; no autonomous confirmation; no autonomous promotion.
Learning	Derived, disposable, event-sourced, rebuild-identical.
System	Offline-first; deterministic; advisory-only decision systems; MCP backward-compatible.
Execution	No autonomous exploitation; no offensive execution; SAFE adapter profiles only.
Federation	Metadata only - never findings, evidence, targets, sources, or secrets.
Marketplace	Declarative plugins only - no plugin execution.
Confidence	No hidden confidence modification or rewriting.

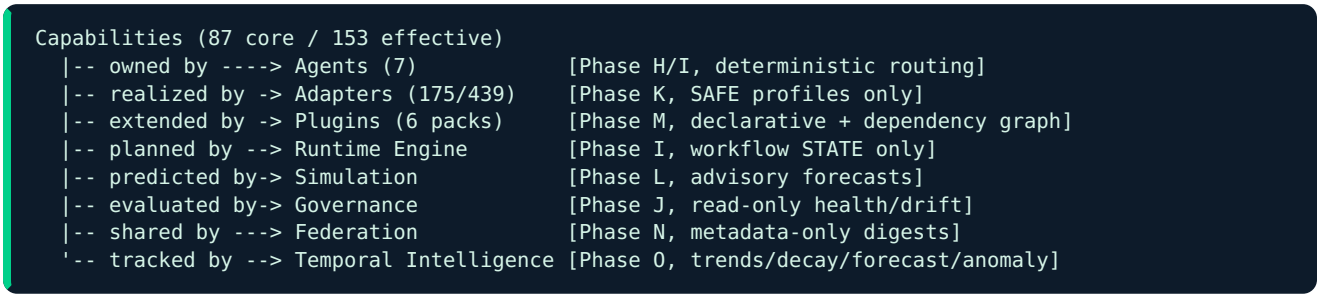
Invariant regression score at Phase O: 100% preserved (0 regressions).

15 Data Flow & Architecture Graph

Data flow map



Architecture relationship graph



16 Performance History & Benchmarks

Every derived layer targets **O(E)** (linear in event count); no O(N²). Capability/MCP growth across phases:

Phase	Caps	Adapters	Agents	MCP	Scaling characteristic
A	87	-	-	10	recon fusion; index rebuild
C.5	87	-	-	16	removed O(F^2) -> O(E)
D/D.1	87	-	-	20	O(E) event-sourced learning
F	87	-	-	26	O(E), WAL idempotent
H	87	-	6	34	deterministic routing
I	87	-	7	38	O(steps) workflow state
K	87	175	7	50	O(E) tool-health
L	87	175	7	58	O(E) load-once SimulationContext
M	153	439	7	70	O(C) incremental plugin loading
N	153	439	7	102	O(E); ~2 us/event federation reads
O	153	439	7	**108**	O(E); 9.35 s @ 1M rows (2M events)

Phase-O temporal benchmark (load-once context, memoized bucketing)

Events (rows)	ctx build	trends	summary	TOTAL	µs/event
10,000	0.04 s	0.30 s	0.27 s	0.61 s	61.5
100,000	0.91 s	0.38 s	0.21 s	1.51 s	15.1
1,000,000	5.96 s	1.78 s	1.61 s	**9.35 s**	**9.35**

Per-event cost **decreases then flattens** (61 -> 15 -> 9 µs) as fixed overhead amortizes — confirming linear O(E) scaling with no performance cliff. 1M rows = 2M derived events (worst case).

Federation benchmark (Phase N)

Read cost flat at **~1.6–2.0 µs/event across 2k–40k events** — O(E), rebuild-identical digests.

17 Open Risks

Risk history is never deleted — items move Open → Mitigated → Closed.

Class	Item	Status
Architectural debt	Branch/tag drift; no root INDEX.md; source_learning vs source_metrics overlap.	Open
Scaling	Mesh/consensus re-read patterns; no event-log compaction beyond ~1e6.	Open
Performance	Per-store SQLite connections, no shared pool (fixed fan-out cost).	Open
Coverage	Exercise coverage is cold-start low until learning stores fill.	Mitigated
Future	Apply load-once context pattern to federation; version plugin deps across peers.	Open

18 Roadmap (Phase O -> Z)

All future phases inherit every A–O invariant: derived, deterministic, offline-first, advisory-only, rebuildable, canonical-wiki-centered.

Phase	Goal	Status	MCP impact
O	Temporal Knowledge Intelligence	DELIVERED (current)	+6 (108)
P	Unified Intelligence & Cross-Store Correlation Layer	next	+~4
Q	Federated Trust Graph & Reputation Hardening	planned	+~4
R	Reporting & Deliverable Synthesis Intelligence	planned	+~3
S	Knowledge Compaction & Snapshotting	planned	+~2
T	Multi-Tenant Scope & Isolation	planned	+~3
U	Observability & Audit Intelligence	planned	+~3
V	Capability Confidence Calibration (advisory)	planned	+~2
W	Federated Simulation Exchange	planned	+~3
X	Adaptive Roadmap & Self-Planning Intelligence	planned	+~2
Y	Cross-Ecosystem Interop & Schema Federation	planned	+~2
Z	Architecture Self-Audit & Invariant Enforcement Engine	planned	+~3

19 Validation & Quality

Dimension	Result
Total tests	499 passing (6 integration/e2e deselected)
Temporal (Phase O)	23 (18 unit + 5 MCP)
Federation (Phase N)	26 (19 unit + 7 MCP)
Determinism	rebuild-identical under injected clock (digest + summary byte-identical)
MCP contract	108 live = 108 baseline = 108 documented; 0 orphans/dupes/missing
Invariants	promotion/confidence immutable; no-wiki-mutation; metadata-only federation
Lint	ruff clean across all phase surfaces

The project follows an **Architecture Steward protocol**: every phase passes a design review, an invariant/safety gate, benchmarks, and an architecture-memory update before it is considered complete.

20 How to Contribute

Contributions are welcome — but HYDRA grows by **preserving invariants**, never by trading architecture for speed.

Development workflow

BASH

```
git checkout -b phase-x-feature
# ... implement (derived, deterministic, advisory) ...
python -m pytest -q            # must stay green (499+)
ruff check .                   # lint clean
# regenerate MCP baseline + update CLAUDE.md if you added tools
git commit -m "..."           # invariant-preserving, scoped commit
```

Adding a new Capability

Declare it in `capabilities/capability_catalog.yaml` (id, category, target/finding types, verification coverage, offline_runnable, tools). Keep ids **globally unique**. No code change is needed for the catalog to pick it up.

Adding a new Adapter

Adapters are synthesized per `capability×tool`. Provide only a **SAFE execution profile** (offline / passive / validation / simulation) with timeouts and I/O schemas — exploitation/persistence/ destructive/weaponized profiles are rejected at load.

Adding a new Plugin

Drop a declarative YAML pack into `hydra/plugins/packs/` (or `$HYDRA_PLUGIN_DIR`): capabilities, adapters, agents, dependencies. Plugins are **declarative data only — never executed**. The dependency graph must stay acyclic.

Adding a new MCP tool

MCP_SERVER.PY

```
@mcp.tool()
def my_tool(arg: str = "") -> str:
    """One-line summary (mirrored into CLAUDE.md)."""
    if (g := _kb_guard()):
        return g
    return json.dumps(MyAnalyzer().report(), indent=2)

# then: regenerate tests/mcp/tool_contract_baseline.json
#       add the tool to CLAUDE.md (doc-sync test is enforced)
```

Guidelines for maintaining invariants

- **Never** modify `promotion.py` or `confidence.py`.
- **Never** add a second canonical source or a dual-write; the wiki is canonical.
- New intelligence layers must be **derived + advisory + deterministic** (inject clocks, sort outputs, key by stable ids, rebuild-identical).
- No autonomous confirmation, promotion, exploitation, or tool execution.
- Federation payloads are **metadata only**; run them through the safety guard.

> New phases follow the **Architecture Steward protocol**: design review → invariant/safety gate → implementation → benchmarks → architecture-memory update (`docs/HYDRA_SYSTEM_CONTEXT.md`).

21 Troubleshooting & FAQ

Question / symptom	Answer
The MCP server starts but knowledge tools return an error.	The knowledge layer failed to import. Tools return `{"success": false, "error": "knowledge layer unavailable: ..."}` — check the traceback, ensure `pip install -r requirements.txt` succeeded and you run from the repo root.
A derived database looks corrupted or stale.	Delete it — every `data/*.db` is disposable and rebuilds identically from its event log. Re-run `kb_rebuild_index` for the graph index.
Recon tools (subfinder/ httpx/...) aren't found.	They are optional and only needed for LIVE recon. Run the `check_tools` MCP tool to see what is installed; the core knowledge OS is fully offline-first without them.
Tests fail on the MCP contract test after I added a tool.	Regenerate `tests/mcp/tool_contract_baseline.json` and add the tool to `CLAUDE.md` — the doc-sync test enforces that code and docs match.
Why won't it promote my hypothesis straight to a pattern?	By design. `FORBIDDEN_PROMOTIONS` blocks skipping validation; a hypothesis must become a finding (two independent signals) before it can inform a pattern or chain.
Can HYDRA exploit a target automatically?	No. There is no autonomous exploitation path; only SAFE adapter profiles load and a human stays in the loop for anything offensive. HYDRA advises and orchestrates.
Temporal tools return nulls / empty results.	Cold start — the learning event logs are empty. Temporal intelligence is derived from them, so trends/forecasts populate as `record_*` events accumulate.
How do I isolate state for testing?	Point the `HYDRA_*_DB` and `HYDRA_WIKI_DIR` env vars at a temp directory; every store and the wiki honor these overrides.

Still stuck? Read `docs/HYDRA_SYSTEM_CONTEXT.md` (architecture memory) and `wiki/SCHEMA.md` (knowledge conventions) — they are the authoritative references.

22 Appendix - Glossary & Maintenance

Term	Meaning
Capability	An abstract objective realized by interchangeable tools.
Adapter	A capability x tool binding with a SAFE execution profile.
Agent	A declarative owner of capability categories that plans & routes (never executes).
Canonical wiki	The single source of truth; only propose-only writers may modify it.
Derived store	An event-sourced, disposable, rebuildable learning/intelligence database.
Two-Signal rule	Confidence elevates only with two independent corroborating signals.
Promotion	Forward-only stage climb: observation->intel->hypothesis->finding->pattern->chain.
MCP	Model Context Protocol - how the LLM operator invokes HYDRA's 108 tools.
Rebuild-identical	Deleting and replaying a derived store yields byte-identical state.

Maintenance protocol

After any phase completes, the architecture memory (`docs/HYDRA_SYSTEM_CONTEXT.md`), `CLAUDE.md`, and the MCP contract baseline are updated automatically with the new counts, benchmarks, tests, risks, and roadmap. The memory file is a first-class architecture artifact.